

Отчёт по лабораторной работе №7

**Дискретно-событийное моделирование: М/М/с и
модель Росса**

Глущенко Евгений Игоревич

2026-05-15

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Система массового обслуживания М/М/с	7
3.2	Модель Росса	8
3.3	Воспроизводимая организация проекта	9
4	Выполнение лабораторной работы	10
4.1	Структура проекта	10
4.2	Основные команды воспроизведения	11
4.3	Реализация модели М/М/с	12
4.4	Базовый эксперимент М/М/с	14
4.5	Параметрическое исследование М/М/с	20
4.6	Реализация модели Росса	23
4.7	Базовый эксперимент модели Росса	25
4.8	Параметрическое исследование модели Росса	33
4.9	Генерация literate-материалов	37
5	Выводы	41
6	Приложение. Команды воспроизведения	42

Список иллюстраций

4.1	Инициализация Julia-проекта lab07	12
4.2	Запуск базового сценария scripts/mmc.jl	15
4.3	Длина очереди в базовом сценарии М/М/с	16
4.4	Число занятых каналов в базовом сценарии М/М/с	17
4.5	Гистограмма времени ожидания в очереди	17
4.6	Сравнение аналитики и имитации для базового сценария М/М/с	18
4.7	Запуск сценария scripts/mmc_parameters.jl	20
4.8	Среднее ожидание по числу каналов	21
4.9	Среднее ожидание по интенсивности потока	22
4.10	Карта загрузки каналов для М/М/с	23
4.11	Запуск сценария scripts/ross.jl	26
4.12	Число исправных машин в демонстрационном прогоне	27
4.13	Число резервных машин во времени	28
4.14	Длина очереди на ремонт	28
4.15	Гистограмма времени до отказа системы	29
4.16	Сравнение аналитической и имитационной оценки времени до отказа	30
4.17	Средняя загрузка ремонтника	31
4.18	Запуск сценария scripts/ross_parameters.jl	33
4.19	Время до отказа по числу рабочих машин	35
4.20	Время до отказа по числу резервов	35
4.21	Сравнение конфигураций по числу ремонтников	36
4.22	Загрузка ремонтников по сценариям	37
4.23	Запуск сценария scripts/tangle.jl	39
4.24	Проверка содержимого каталогов результатов	40

Список таблиц

4.1	Сводка базового эксперимента М/М/с	15
4.2	Первые строки <code>mmc_customers.csv</code>	19
4.3	Начало журнала событий <code>mmc_events.csv</code>	19
4.4	Выбранные строки <code>mmc_parameter_scan.csv</code>	20
4.5	Сводка базового эксперимента модели Росса	27
4.6	Начало журнала <code>ross_events_sample.csv</code>	31
4.7	Выбранные строки <code>ross_runs.csv</code>	32
4.8	Сценарии <code>ross_parameter_scan.csv</code> при $S = 1$	33
4.9	Сценарии <code>ross_parameter_scan.csv</code> при $S = 3$	34

1 Цель работы

Изучить дискретно-событийное моделирование на примере системы массового обслуживания М/М/с и модели отказов с ремонтом Росса, реализовать оба сценария на Julia, провести базовые и параметрические эксперименты, а также подготовить воспроизводимые отчётные материалы.

Исполнитель работы: Глущенко Евгений Игоревич.

Группа: НФИбд-01-23.

Студенческий билет: 1132239110.

2 Задание

В лабораторной работе требовалось:

1. Создать рабочий каталог и оформить проект в структуре DrWatson.
2. Реализовать модель М/М/с и модель Росса на Julia.
3. Перенести сценарии в literate-формат.
4. Сформировать clean-скрипты, markdown-документы и ipynb-ноутбуки.
5. Для М/М/с построить графики длины очереди, числа занятых каналов, гистограмму ожидания и сравнение с аналитикой.
6. Для модели Росса реализовать несколько ремонтников, собрать статистику времени до отказа и сравнить моделирование с аналитическим расчётом.
7. Выполнить параметрические исследования и включить их результаты в отчёт.

3 Теоретическое введение

3.1 Система массового обслуживания М/М/с

Система М/М/с относится к классическим моделям теории массового обслуживания [1]. Обозначение М/М/с означает:

- входящий поток заявок является пуассоновским с интенсивностью λ ;
- времена обслуживания независимы и имеют экспоненциальное распределение с параметром μ ;
- обслуживание выполняется с параллельными каналами;
- дисциплина обслуживания — FIFO.

Коэффициент загрузки системы определяется как

$$\rho = \frac{\lambda}{c\mu}.$$

При $\rho < 1$ существуют стационарные характеристики, которые в работе вычисляются через формулы Эрланга:

$$a = \frac{\lambda}{\mu},$$

$$P_0 = \left(\sum_{n=0}^{c-1} \frac{a^n}{n!} + \frac{a^c}{c!(1-\rho)} \right)^{-1},$$

$$P_{wait} = \frac{a^c}{c!(1-\rho)} P_0,$$

$$L_q = P_{wait} \frac{\rho}{1-\rho}, \quad W_q = \frac{L_q}{\lambda}, \quad W = W_q + \frac{1}{\mu}, \quad L = \lambda W.$$

Эти значения используются как аналитическая база для сравнения с имитационным экспериментом.

3.2 Модель Росса

Модель Росса рассматривает систему из N работающих машин и S резервных машин. Отказы происходят случайно, а неисправные элементы восстанавливаются ремонтниками. Если в момент отказа свободных резервов уже нет, система считается отказавшей [2].

В реализации работы приняты следующие допущения:

- каждая рабочая машина имеет одинаковое среднее время до отказа τ_f ;
- время ремонта подчиняется экспоненциальному распределению со средним τ_r ;
- число ремонтников равно $repairers$;
- одновременно может выполняться не более $repairers$ ремонтов, остальные неисправности ждут в очереди.

Для аналитической оценки используется система линейных уравнений по числу доступных резервов $k = 0, \dots, S$:

$$\alpha = \frac{N}{\tau_f}, \quad \beta_k = \frac{\min(repairers, S - k)}{\tau_r},$$

$$(\alpha + \beta_k)T_k = 1 + \alpha T_{k-1} + \beta_k T_{k+1},$$

где T_k — ожидаемое время до отказа системы при наличии k резервов, а граничное условие $T_{-1}=0$ соответствует уже наступившему отказу. В коде эта система решается как линейная алгебраическая задача.

3.3 Воспроизводимая организация проекта

Структура вычислительного проекта построена на `DrWatson.jl` [3]. Для генерации производных сценариев используются `Literate.jl` [4], а сами расчёты выполнены на языке Julia [5]. Такой подход позволяет поддерживать единый источник сценария и автоматически получать:

- исполняемый `clean`-скрипт;
- `markdown`-документ;
- Jupyter notebook;
- итоговые графики и таблицы.

4 Выполнение лабораторной работы

4.1 Структура проекта

Рабочий каталог лабораторной работы имеет следующий состав:

- `project/src/QueueingModels.jl` — модуль с аналитическими формулами, моделями и графиками;
- `project/scripts/mmc.jl` — базовый сценарий М/М/с;
- `project/scripts/mmc_parameters.jl` — параметрический сценарий М/М/с;
- `project/scripts/ross.jl` — базовый сценарий модели Росса;
- `project/scripts/ross_parameters.jl` — параметрический сценарий модели Росса;
- `project/scripts/tangle.jl` — генератор literate-артефактов;
- `project/data/` — таблицы результатов;
- `project/plots/` — итоговые графики;
- `project/docs/` — сгенерированные markdown-материалы;
- `project/notebooks/` — сгенерированные ipynb-файлы;
- `report/` и `presentation/` — оформление отчёта и презентации на Quarto.

Число сформированных основных артефактов:

- `mmc_customers.csv` — 5000 строк данных о заявках;
- `mmc_events.csv` — 15000 событий;
- `mmc_parameter_scan.csv` — 18 сценариев;
- `ross_events_sample.csv` — 10946 событий демонстрационного прогона;
- `ross_runs.csv` — 300 независимых запусков;
- `ross_parameter_scan.csv` — 12 сценариев;
- `docs/` — 4 markdown-файла;
- `notebooks/` — 4 исполняемых notebook-файла.

4.2 Основные команды воспроизведения

Базовая последовательность команд для выполнения лабораторной работы выглядит так:

```
cd ~/labs/lab07/project
~/juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
~/juliaup/bin/julia --project=. -e 'include("scripts/mmc.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/mmc_parameters.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/ross.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/ross_parameters.jl")'
~/juliaup/bin/julia --project=. scripts/tangle.jl
```

Отдельный локальный файл с командами для записи видео вынесен в `commands-lab07.txt`.

4.2.1 Фактическое выполнение команд

На Рисунок 4.1 показан переход в каталог проекта и активация зависимостей Julia через `Pkg.instantiate()`.

```

ltdj@ltdj@eiglushchenko:~$ cd ~/labs/lab07/project
ltdj@ltdj@eiglushchenko:~/labs/lab07/project$ ~/.juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
Precompiling packages finished.
1 dependency successfully precompiled in 8 seconds. 308 already precompiled.

```

Рисунок 4.1: Инициализация Julia-проекта lab07

Скриншот подтверждает, что окружение проекта было корректно поднято, а недостающие зависимости доустановлены и предварительно скомпилированы. После этого все последующие сценарии запускались уже в одном и том же воспроизводимом окружении.

4.3 Реализация модели M/M/c

4.3.1 Аналитические характеристики

Вычисление стационарных характеристик вынесено в функцию `analytic_mmc_metrics`:

```

function analytic_mmc_metrics(lambda::Real, mu::Real, c::Integer)
    lambda_f = Float64(lambda)
    mu_f = Float64(mu)
    rho = lambda_f / (c * mu_f)
    if rho >= 1
        return (rho = rho, p0 = NaN, pwait = NaN, lq = Inf, wq = Inf, w = Inf)
    end

    a = lambda_f / mu_f
    sum_terms = sum((a^n) / factorial(big(n)) for n in 0:(c - 1))
    tail_term = (a^c) / (factorial(big(c)) * (1 - rho))
    p0 = 1 / Float64(sum_terms + tail_term)
    pwait = Float64((a^c) / (factorial(big(c)) * (1 - rho))) * p0
    lq = pwait * rho / (1 - rho)
end

```

```

wq = lq / lambda_f
w = wq + 1 / mu_f
l = lambda_f * w
return (rho = rho, p0 = p0, pwait = pwait, lq = lq, wq = wq, w = w, l =
end

```

Если $\rho \geq 1$, функция корректно помечает сценарий как нестационарный через бесконечные значения характеристик очереди.

4.3.2 Имитационный алгоритм

Сама модель М/М/с реализована на уровне событий прибытия, начала обслуживания и выхода из системы:

```

if process_arrival
  customer_id = next_customer_id
  next_customer_id += 1
  time = next_arrival
  arrival_time[customer_id] = time
  service_time[customer_id] = rand(rng, service_dist)

  if busy_servers < c
    sid = findfirst(!, server_busy)
    server_busy[sid] = true
    busy_servers += 1
    service_start[customer_id] = time
    wait_time[customer_id] = 0.0
    departure_time[customer_id] = time + service_time[customer_id]
  else
    push!(queue, customer_id)
  end
end

```

```

        end
    else
        time = next_departure
        customer_id = server_customer[departure_server]
        departed_customers += 1
        server_busy[departure_server] = false
        busy_servers -= 1
        ...
    end
end

```

Дополнительно в реализации введён параметр `warmup_customers`, позволяющий исключить начальный переходный участок из расчёта средних значений.

4.4 Базовый эксперимент M/M/c

Для базового сценария использованы параметры:

- `lambda = 0.9;`
- `mu = 0.5;`
- `c = 2;`
- `num_customers = 5000;`
- `seed = 123;`
- `warmup_customers = 500.`

Сценарий `scripts/mmc.jl` формирует:

- `data/mmc_customers.csv;`
- `data/mmc_events.csv;`
- `data/mmc_summary.csv;`

- plots/mmc_queue_length.png;
- plots/mmc_busy_servers.png;
- plots/mmc_wait_histogram.png;
- plots/mmc_analytic_vs_simulation.png.

Вывод базового запуска:

```
M/M/c baseline experiment completed
customers: 5000
mean wait: 8.3449
utilization: 0.8989
```

На Рисунок 4.2 показан фактический запуск базового сценария М/М/с в терминале.

```
lildji@eiglushchenko:~/labs/lab07/project$ ~/.juliaup/bin/julia --project=. -e 'include("scripts/mmc.jl")'
M/M/c baseline experiment completed
customers: 5000
mean wait: 8.3449
utilization: 0.8989
```

Рисунок 4.2: Запуск базового сценария scripts/mmc.jl

Основные численные результаты приведены в Таблица 4.1.

Таблица 4.1: Сводка базового эксперимента М/М/с

lambda	mu	c	rho	analytic_mean_wait	sim_mean_wait	analytic_max_wait	sim_max_wait	analytic_utilization	sim_utilization
0.9	0.5	2	0.9	8.526	8.345	7.674	7.618	0.853	0.856
									0.899

На Рисунок 4.3 показана длина очереди во времени.

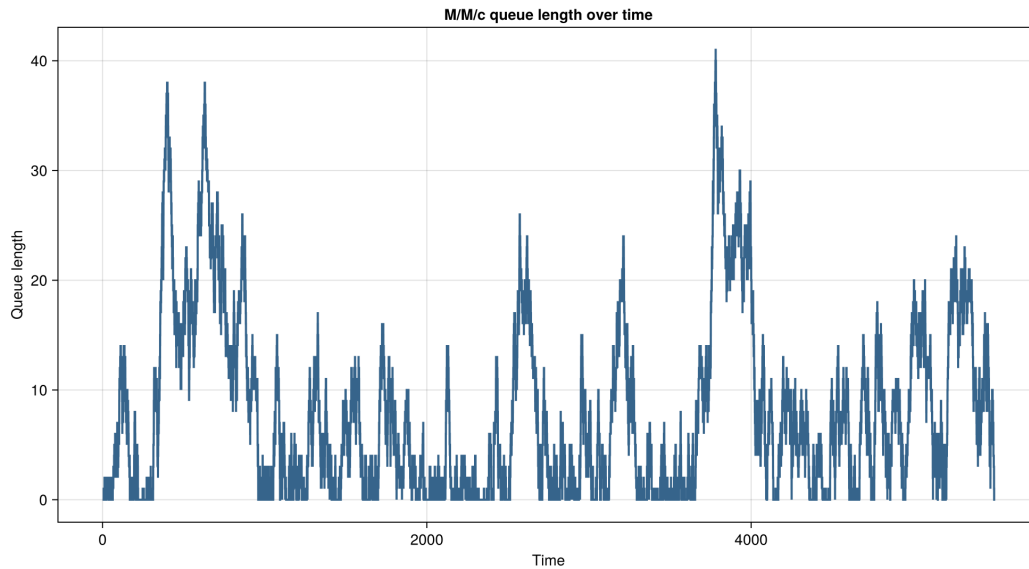


Рисунок 4.3: Длина очереди в базовом сценарии М/М/с

График демонстрирует выраженные колебания очереди при высокой загрузке $\rho = 0.9$. Очередь не расходится, но регулярно принимает двузначные значения, что соответствует работе системы вблизи границы устойчивости.

На Рисунок 4.4 показано число занятых каналов обслуживания.

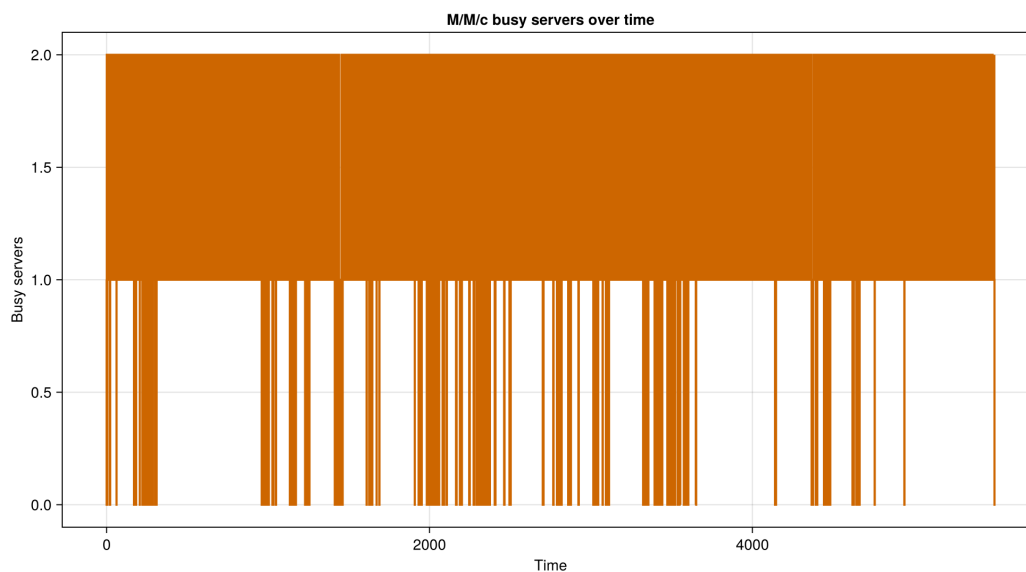


Рисунок 4.4: Число занятых каналов в базовом сценарии М/М/с

Большую часть времени оба канала заняты. Это согласуется с оценкой загрузки 0.8989, то есть почти 90% полезного времени каждый канал находится в работе.

На Рисунок 4.5 показана гистограмма времени ожидания.

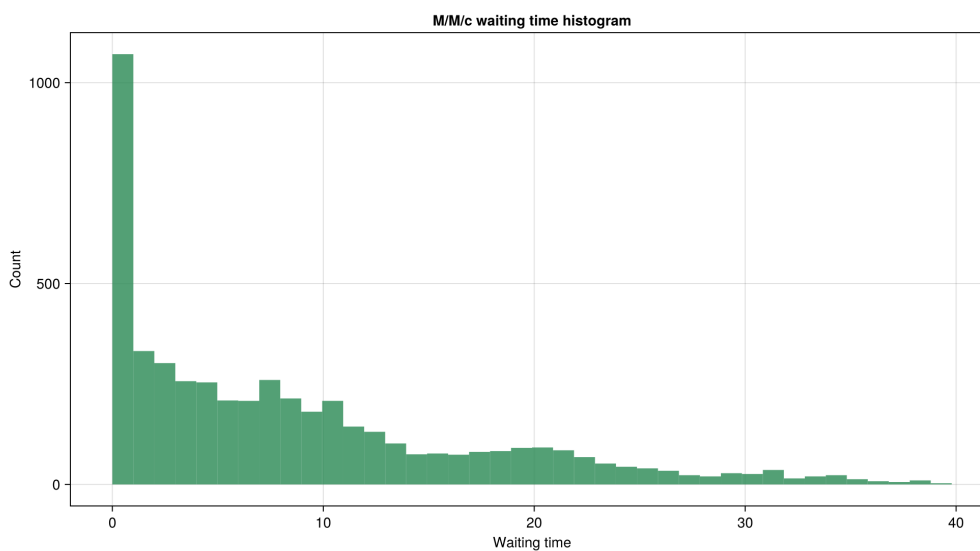


Рисунок 4.5: Гистограмма времени ожидания в очереди

Распределение ожидания имеет длинный правый хвост: заметная часть заявок обслуживается быстро, но в периоды перегрузки отдельные заявки проводят в очереди существенное время.

На Рисунок 4.6 показано сравнение аналитических и имитационных характеристик.

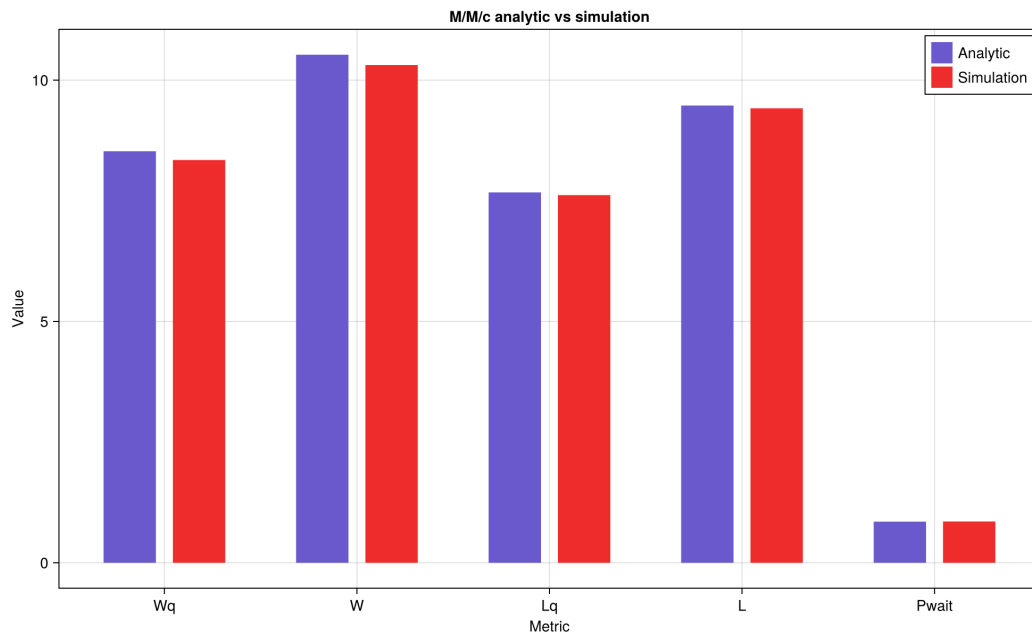


Рисунок 4.6: Сравнение аналитики и имитации для базового сценария M/M/c

Для базового сценария имитационные оценки хорошо согласуются с аналитикой. Это особенно важно для W_q и L_q , поскольку именно эти величины наиболее чувствительны к режиму загрузки системы.

4.4.1 Фрагмент таблицы заявок mmc_customers.csv

Первые строки таблицы заявок приведены в Таблица 4.2.

Таблица 4.2: Первые строки mmc_customers.csv

id	arrival_time	service_start_time	departure_time	wait_time	service_time	seem_time	server_id
1	0.145	0.145	0.953	0.000	0.808	0.808	1
2	1.100	1.100	1.235	0.000	0.135	0.135	1
3	1.167	1.167	2.261	0.000	1.094	1.094	2
4	3.246	3.246	4.663	0.000	1.417	1.417	1
5	5.091	5.091	6.565	0.000	1.474	1.474	1
6	6.233	6.233	11.540	0.000	5.307	5.307	2

4.4.2 Фрагмент журнала событий mmc_events.csv

Начало журнала событий приведено в Таблица 4.3.

Таблица 4.3: Начало журнала событий mmc_events.csv

time	event	customer_id	queue_length	busy_servers	system_size
0.145	arrival	1	0	1	1
0.145	service_start	1	0	1	1
0.953	departure	1	0	0	0
1.100	arrival	2	0	1	1
1.100	service_start	2	0	1	1
1.167	arrival	3	0	2	2
1.167	service_start	3	0	2	2
1.235	departure	2	0	1	1

Журнал отражает трёхтипную логику модели: прибытие заявки, начало обслуживания и завершение обслуживания. На раннем участке очереди ещё нет, а оба канала заполняются по мере поступления заявок.

4.5 Параметрическое исследование М/М/с

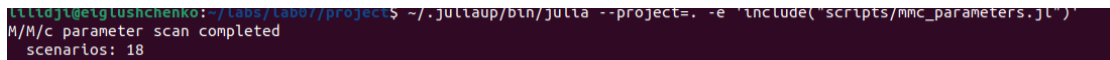
В сценарии `scripts/mmc_parameters.jl` исследование выполняется на сетке:

- `lambdas = [0.3, 0.6, 0.9];`
- `channels = 1:6;`
- `mu = 0.5;`
- `num_customers = 3000;`
- `seed = 321.`

Вывод сценария:

```
M/M/c parameter scan completed
scenarios: 18
```

На Рисунок 4.7 показан запуск параметрического исследования М/М/с.



```
ttidj@getglushchenko: ~/labs/lab07/projects$ ~/.juliaup/bin/julia --project=. -e 'include("scripts/mmc_parameters.jl")'
M/M/c parameter scan completed
scenarios: 18
```

Рисунок 4.7: Запуск сценария `scripts/mmc_parameters.jl`

Выбранные строки таблицы `mmc_parameter_scan.csv` приведены в Таблица 4.4.

Таблица 4.4: Выбранные строки `mmc_parameter_scan.csv`

lambda	c	rho	analytic_wq	sim_wq	sim_utilization
0.3	1	0.6	3.000	3.078	0.579
0.3	2	0.3	0.198	0.194	0.294
0.3	3	0.2	0.021	0.021	0.204
0.6	1	1.2	Inf	333.856	0.999

lambda	c	rho	analytic_wq	sim_wq	sim_utilization
0.6	2	0.6	1.125	1.248	0.610
0.6	3	0.4	0.157	0.159	0.410
0.9	1	1.8	Inf	1422.276	1.000
0.9	2	0.9	8.526	4.279	0.859
0.9	3	0.6	0.591	0.650	0.592

Для сценариев с $c = 1$ и $\lambda \geq 0.6$ система становится неустойчивой, что сразу отражается как в аналитических значениях, так и в эмпирическом росте среднего времени ожидания. Для случая $\lambda = 0.9$, $c = 2$ заметна более сильная дисперсия оценки W_q , что естественно для режима, близкого к критической нагрузке.

На Рисунок 4.8 показано изменение среднего времени ожидания по числу каналов.

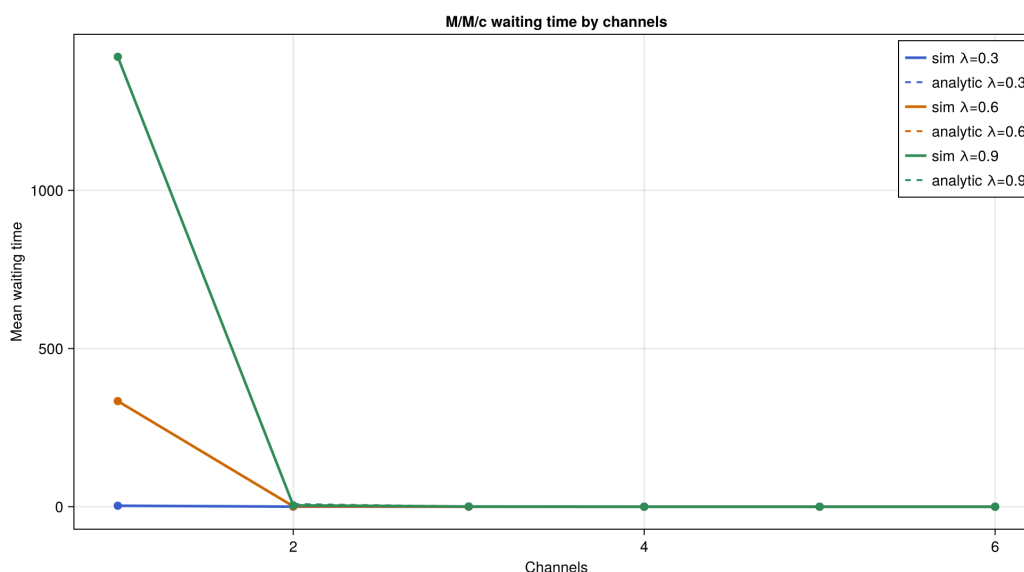


Рисунок 4.8: Среднее ожидание по числу каналов

Добавление каналов резко снижает время ожидания, особенно в

области от $c = 1$ к $c = 2$. Далее выигрыш сохраняется, но становится менее драматичным.

На Рисунок 4.9 показано изменение ожидания по интенсивности входящего потока.

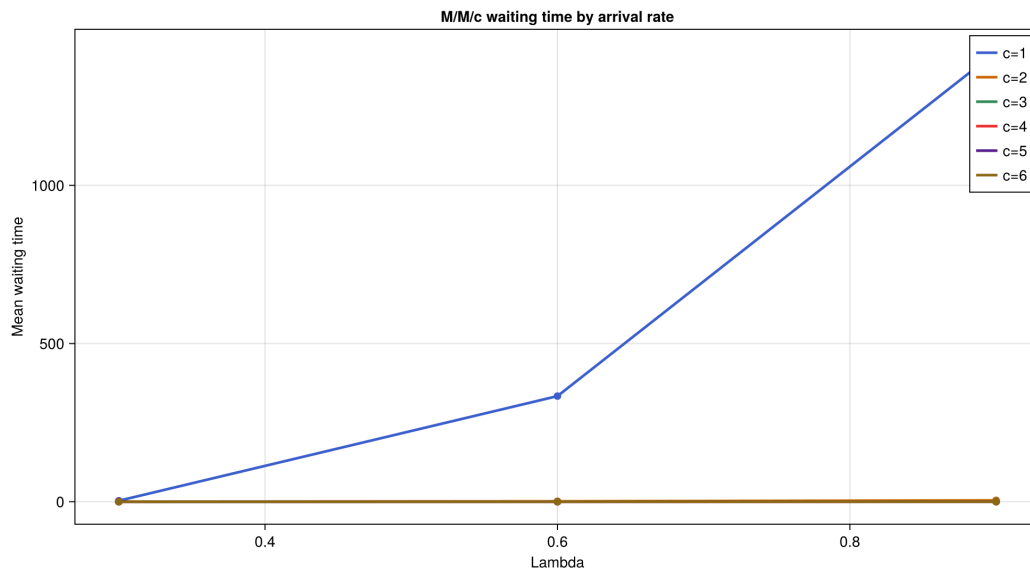


Рисунок 4.9: Среднее ожидание по интенсивности потока

Рост λ при фиксированном m быстро переводит систему в зону длинных очередей. Особенно чувствительны конфигурации с малым числом каналов.

На Рисунок 4.10 показана карта загрузки каналов.

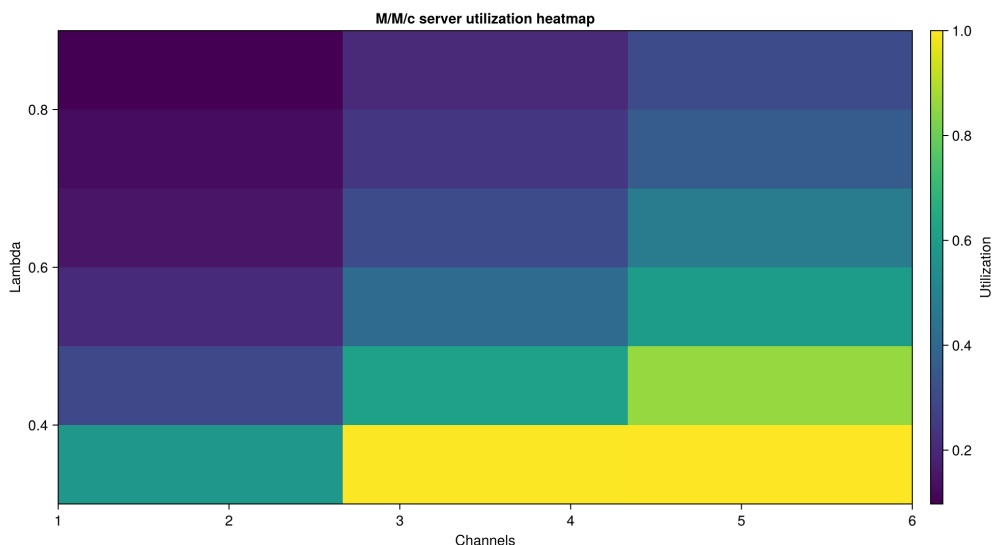


Рисунок 4.10: Карта загрузки каналов для М/М/с

Тепловая карта подтверждает ожидаемую закономерность: при фиксированном потоке загрузка убывает по мере роста s , а при фиксированном числе каналов возрастает вместе с λ .

4.6 Реализация модели Росса

4.6.1 Аналитическая часть

Оценка среднего времени до отказа системы реализована через решение линейной системы:

```
function analytic_ross_crash_time(N::Integer, S::Integer, repairers::Integer,
                                   mean_time_to_failure::Real, mean_repair_time::Real)
    failure_rate = N / Float64(mean_time_to_failure)
    nstates = S + 1
    A = zeros(Float64, nstates, nstates)
    b = ones(Float64, nstates)
```

```

for k in 0:S
    row = k + 1
    repair_rate = min(repairers, S - k) / Float64(mean_repair_time)
    A[row, row] = failure_rate + repair_rate
    if k > 0
        A[row, row - 1] = -failure_rate
    end
    if k < S
        A[row, row + 1] = -repair_rate
    end
end

expected = A \ b
return expected[end]
end

```

Такая реализация хорошо согласуется с постановкой через марковскую цепь по числу доступных резервов.

4.6.2 Имитационный цикл отказов и ремонтов

Ключевая часть событийного алгоритма выглядит следующим образом:

```

if next_failure < next_repair - 1e-12
    time = next_failure
    if spares > 0
        spares -= 1
        broken += 1
    end
end

```

```

        if busy_repairers < repairers
            busy_repairers += 1
            push!(active_repairs, time + rand(rng, Exponential(mean_repair_
        else
            repair_queue += 1
        end
    else
        working -= 1
        broken += 1
        push!(events_internal, (... , event = "crash", ...))
        break
    end
else
    repair_idx = argmin(active_repairs)
    time = active_repairs[repair_idx]
    broken -= 1
    spares += 1
    ...
end

```

Таким образом модель различает два типа событий: отказ машины и завершение ремонта. Отказ без свободного резерва завершает эксперимент.

4.7 Базовый эксперимент модели Росса

Для базового сценария использованы параметры:

- $N = 10$;

- `S = 3;`
- `repairers = 1;`
- `mean_time_to_failure = 100.0;`
- `mean_repair_time = 1.0;`
- `runs = 300;`
- `seed = 150.`

Сценарий `scripts/ross.jl` формирует:

- `data/ross_events_sample.csv;`
- `data/ross_runs.csv;`
- `data/ross_summary.csv;`
- `plots/ross_good_machines.png;`
- `plots/ross_spares.png;`
- `plots/ross_repair_queue.png;`
- `plots/ross_crash_time_histogram.png;`
- `plots/ross_simulation_vs_analytic.png;`
- `plots/ross_repairer_utilization.png.`

Вывод базового запуска:

```
Ross model baseline experiment completed
mean crash time: 11937.8165
analytic crash time: 12340.0
```

На Рисунок 4.11 показан фактический запуск базового сценария модели Росса.

```
lilidjl@eiglushchenko:~/labs/lab07/project$ ~/.juliaup/bin/julia --project=. -e 'include("scripts/ross.jl")'
Ross model baseline experiment completed
mean crash time: 11937.8165
analytic crash time: 12340.0
```

Рисунок 4.11: Запуск сценария `scripts/ross.jl`

Итоговая сводка приведена в Таблица 4.5.

Таблица 4.5: Сводка базового эксперимента модели Росса

N	S	repairers	runs	mean_crash_time	std_crash_time	mean_busy_time	std_busy_time	mean_repair_time	std_repair_time	utilization
10	3	1	300	11937.816	916.152	340.000	120	0.10	10	

На Рисунок 4.12 показано число исправных машин во времени.

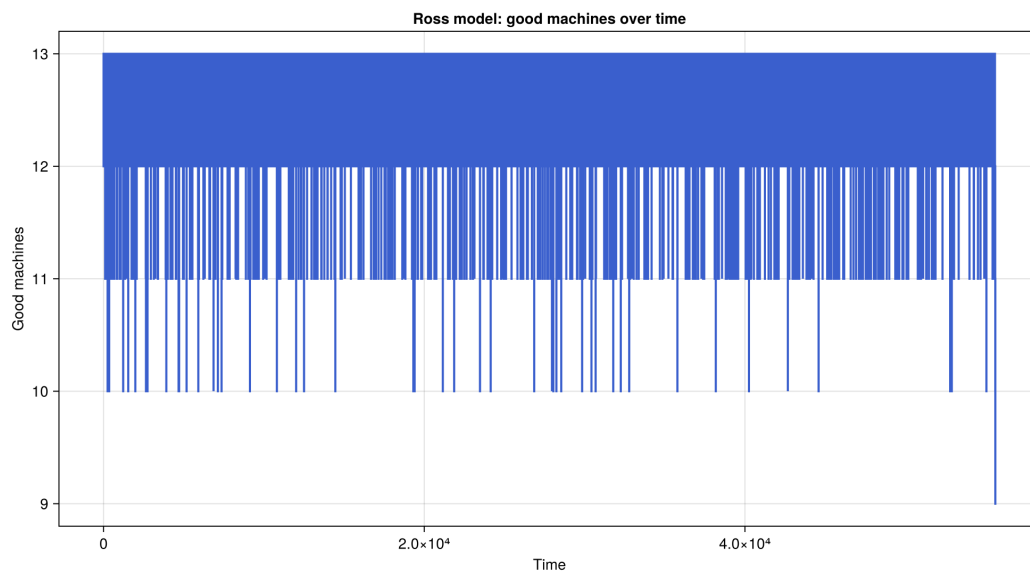


Рисунок 4.12: Число исправных машин в демонстрационном про-
гоне

Пока резервы доступны, число исправных машин удерживается на высоком уровне. Непосредственно перед отказом системы траектория резко уходит вниз.

На Рисунок 4.13 показана динамика резерва.

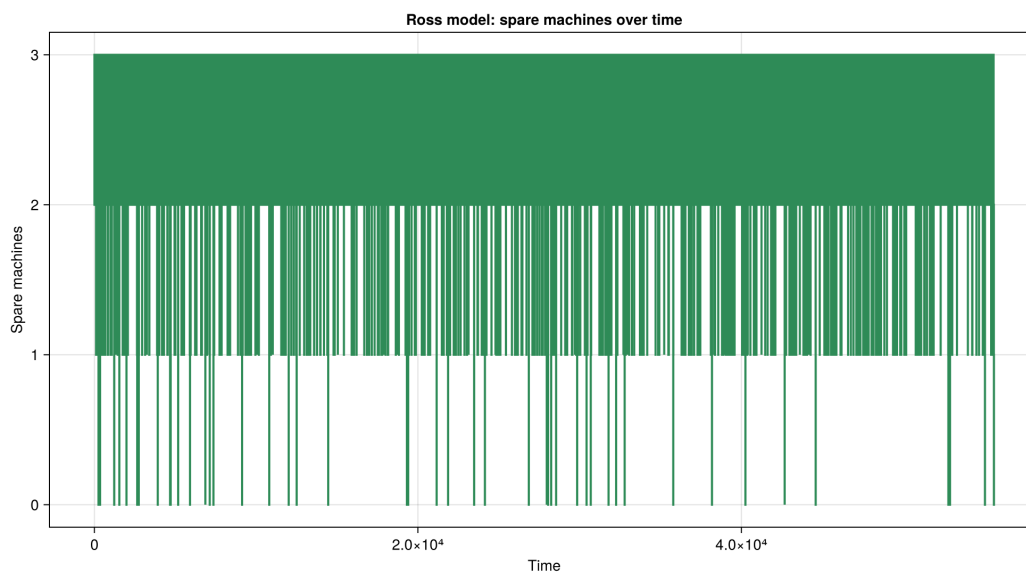


Рисунок 4.13: Число резервных машин во времени

Резерв регулярно расходуется и пополняется за счёт завершения ремонтов. Именно состояние резерва служит главным индикатором близости системного отказа.

На Рисунок 4.14 показана длина очереди на ремонт.

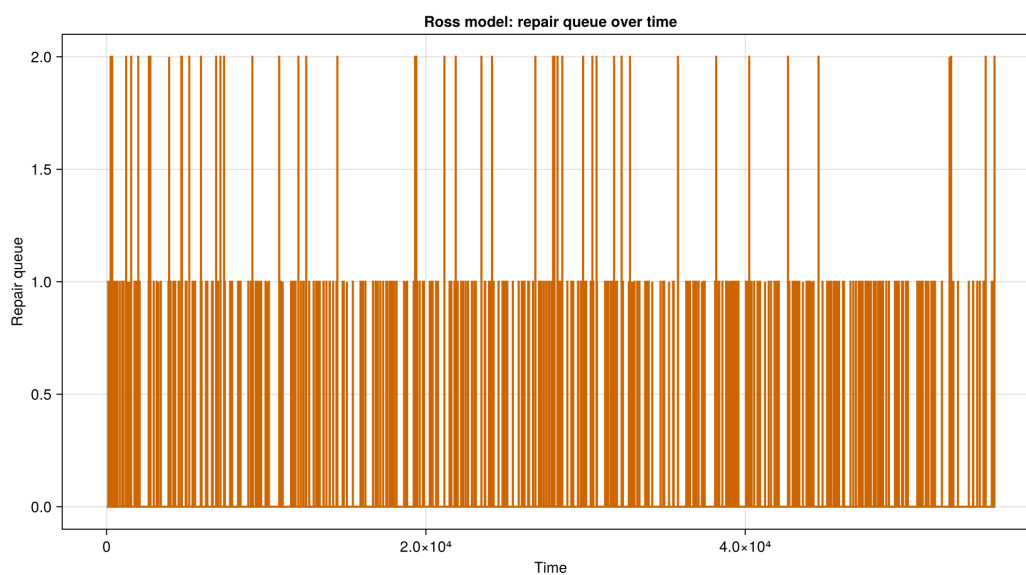


Рисунок 4.14: Длина очереди на ремонт

В базовом сценарии очередь почти всегда близка к нулю, что хорошо согласуется со средним значением 0.0120.

На Рисунок 4.15 показано распределение времени до отказа по 300 независимым прогонам.

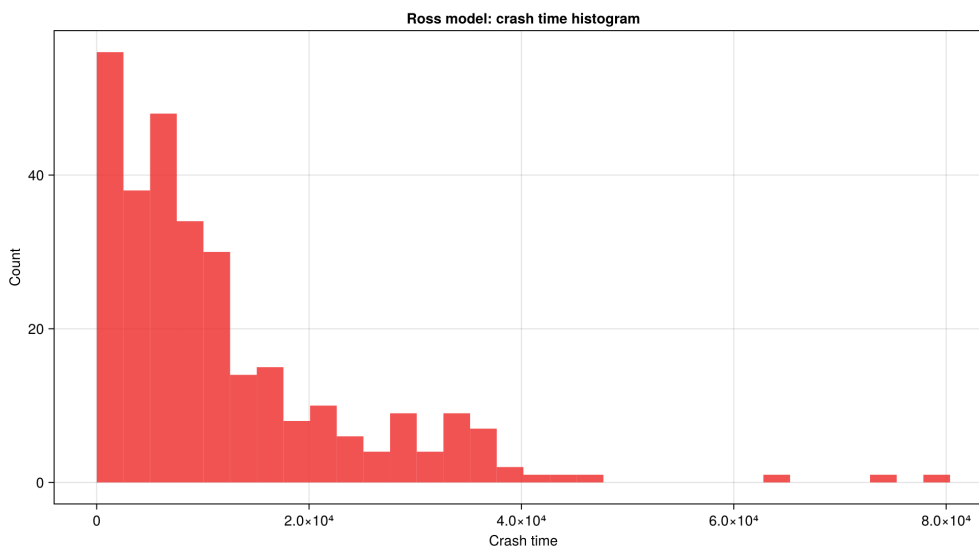


Рисунок 4.15: Гистограмма времени до отказа системы

Распределение заметно асимметрично и имеет длинный хвост. Это ожидаемо: часть прогонов завершается быстро, а часть продолжает работу значительно дольше среднего.

На Рисунок 4.16 показано сравнение имитационной и аналитической оценки времени до отказа.

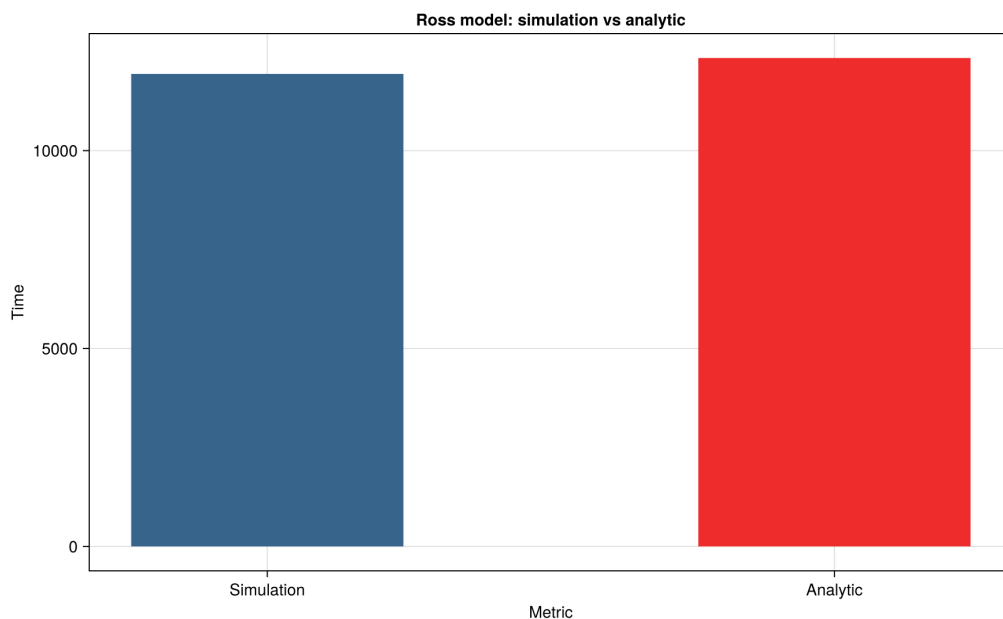


Рисунок 4.16: Сравнение аналитической и имитационной оценки времени до отказа

Имитационное среднее 11937.8165 близко к аналитическому значению 12340.0. Разница порядка нескольких процентов укладывается в наблюдаемую дисперсию прогонов.

На Рисунок 4.17 показана средняя загрузка ремонтника.

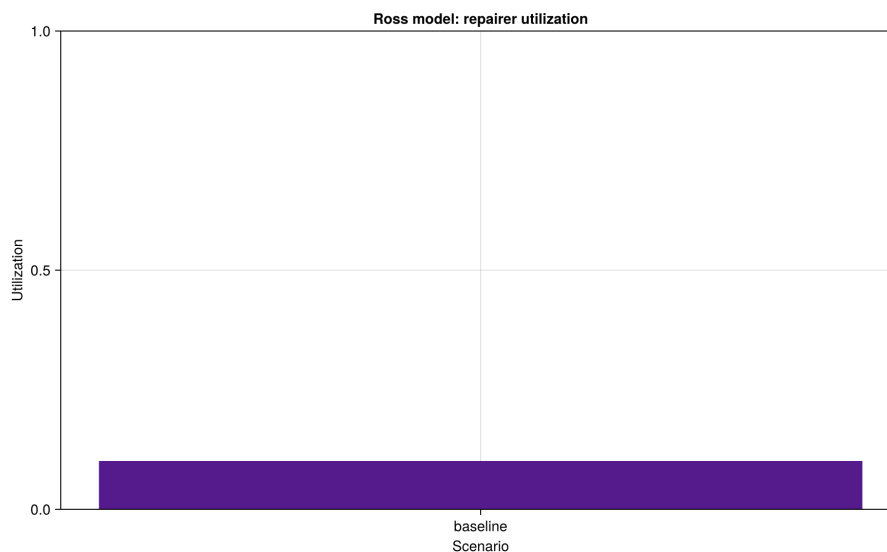


Рисунок 4.17: Средняя загрузка ремонтника

Полученная загрузка 0.1010 указывает, что единственный ремонтник большую часть времени свободен. Именно поэтому добавление второго ремонтника не всегда даёт столь же сильный выигрыш, как добавление резервных машин.

4.7.1 Фрагмент журнала событий

ross_events_sample.csv

Первые строки демонстрационного прогона приведены в Таблица 4.6.

Таблица 4.6: Начало журнала ross_events_sample.csv

time	event	working	spares	broken	repair_queue	busy_repairers	total_machines
1.947	failure	10	2	1	0	1	12
4.396	repair_complete	10	3	0	0	0	13
33.924	failure	10	2	1	0	1	12

time	event	working	spares	broken	repair_queue	busy_repairers	total_machines
34.126	repair_complete	10	3	0	0	0	13
35.593	failure	10	2	1	0	1	12
37.052	repair_complete	10	3	0	0	0	13
49.323	failure	10	2	1	0	1	12
50.985	repair_complete	10	3	0	0	0	13

4.7.2 Фрагмент таблицы прогонов `ross_runs.csv`

Некоторые независимые прогоны приведены в Таблица 4.7.

Таблица 4.7: Выбранные строки `ross_runs.csv`

run_id	crash_time	mean_repair_queue	repairer_utilization
1	1260.232	0.0066	0.0829
2	20307.875	0.0099	0.0981
3	6130.477	0.0070	0.0968
4	1766.511	0.0159	0.0985
5	2450.059	0.0171	0.0942
6	2760.901	0.0084	0.1013

Даже на первых шести прогонах видно, насколько велик разброс времени до отказа. Это и обуславливает необходимость усреднения по многим реализациям.

4.8 Параметрическое исследование модели Росса

В сценарии `scripts/ross_parameters.jl` использована сетка:

- `N_values = [5, 10, 15];`
- `S_values = [1, 3];`
- `repairer_values = [1, 2];`
- `mean_time_to_failure = 100.0;`
- `mean_repair_time = 1.0;`
- `runs = 20;`
- `seed = 500.`

Вывод сценария:

```
Ross parameter scan completed
scenarios: 12
```

На Рисунок 4.18 показан запуск параметрического исследования модели Росса.

```
l11d31@eiglushchenko:~/labs/lab07/project$ ~/.juliaup/bin/julia --project=. -e 'include("scripts/ross_parameters.jl")'
Ross model parameter scan completed
scenarios: 12
l11d31@eiglushchenko:~/labs/lab07/project$ ~/.juliaup/bin/julia --project=. scripts/table_4l
```

Рисунок 4.18: Запуск сценария `scripts/ross_parameters.jl`

Сначала рассмотрим сценарии с одним резервом $S = 1$ (Таблица 4.8).

Таблица 4.8: Сценарии `ross_parameter_scan.csv` при $S = 1$

N	repairers	mean_crash_time	analytic_crash_time	repairer_utilization
5	1	438.678	440.000	0.0724

N	repairers	mean_crash_time	analytic_crash_time	repairer_utilization
5	2	349.707	440.000	0.0221
10	1	115.946	120.000	0.0873
10	2	110.977	120.000	0.0520
15	1	59.630	57.778	0.1250
15	2	34.712	57.778	0.0808

При одном резерве система очень чувствительна к увеличению числа рабочих машин N : время до отказа быстро уменьшается. Добавление второго ремонтника помогает, но не всегда радикально меняет картину, поскольку запас резерва остаётся малым.

Теперь рассмотрим сценарии с тремя резервами $S = 3$ (Таблица 4.9).

Таблица 4.9: Сценарии ross_parameter_scan.csv при $S = 3$

N	repairers	mean_crash_time	analytic_crash_time	repairer_utilization
5	1	182977.323	177280.000	0.0498
5	2	431297.387	690080.000	0.0250
10	1	13106.329	12340.000	0.1014
10	2	39111.248	46540.000	0.0497
15	1	2602.452	2727.901	0.1498
15	2	9349.971	9927.901	0.0784

Увеличение числа резервов радикально повышает устойчивость системы. Это видно особенно на конфигурации $N = 5$: переход от $S = 1$ к $S = 3$ увеличивает время до отказа на несколько порядков.

На Рисунок 4.19 показано влияние числа рабочих машин на время до отказа.

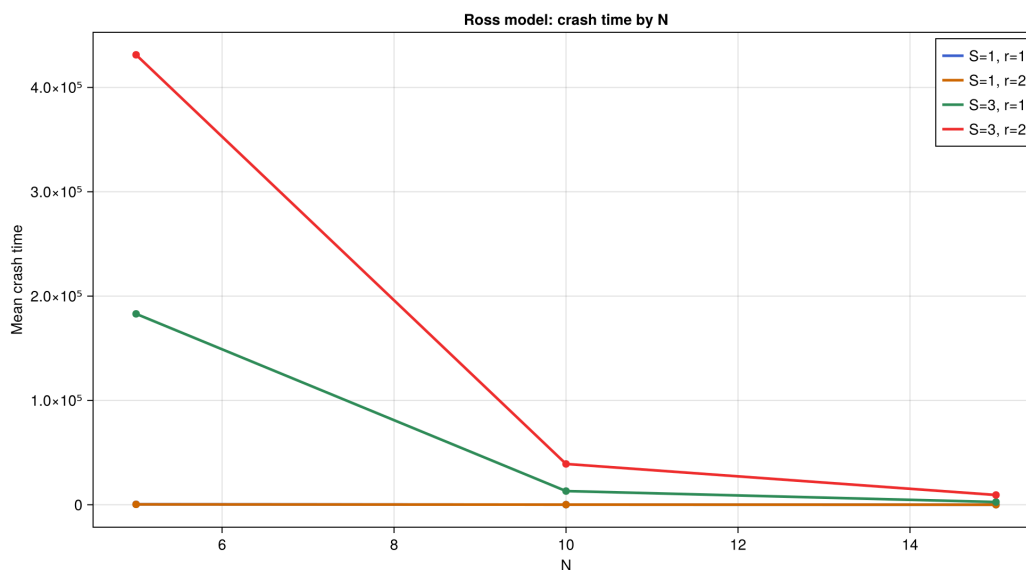


Рисунок 4.19: Время до отказа по числу рабочих машин

При фиксированном резерве рост N ускоряет расходование запаса и сокращает среднее время жизни системы. Эффект сильнее заметен в конфигурациях с малым числом резервов.

На Рисунок 4.20 показано сравнение по числу резервов.

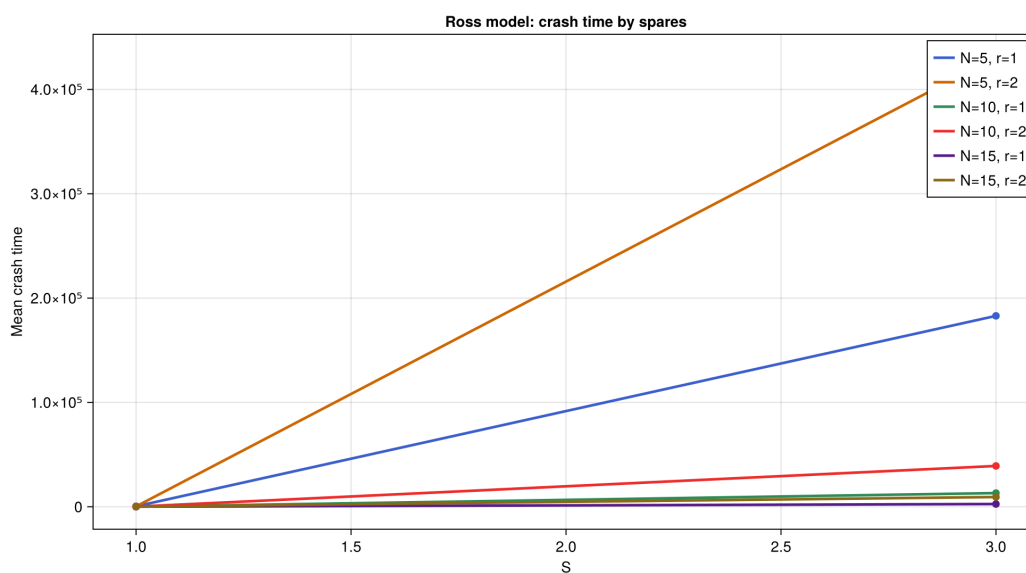


Рисунок 4.20: Время до отказа по числу резервов

Добавление резервных машин является самым эффективным способом увеличить устойчивость. Это согласуется с самой логикой модели: резерв отодвигает момент, когда очередной отказ приведёт к системной аварии.

На Рисунок 4.21 показано сравнение числа ремонтников.

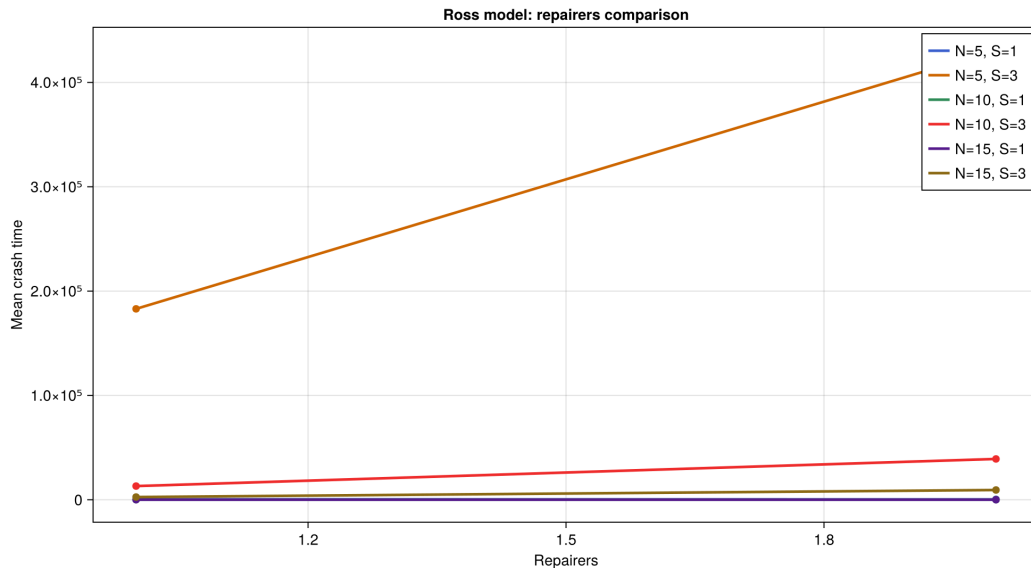


Рисунок 4.21: Сравнение конфигураций по числу ремонтников

Второй ремонтник особенно полезен там, где система действительно успевает накопить повреждённые элементы в ремонте. Если же резерв и так почти не исчерпывается, эффект выражен слабее.

На Рисунок 4.22 показана загрузка ремонтников по сценариям.

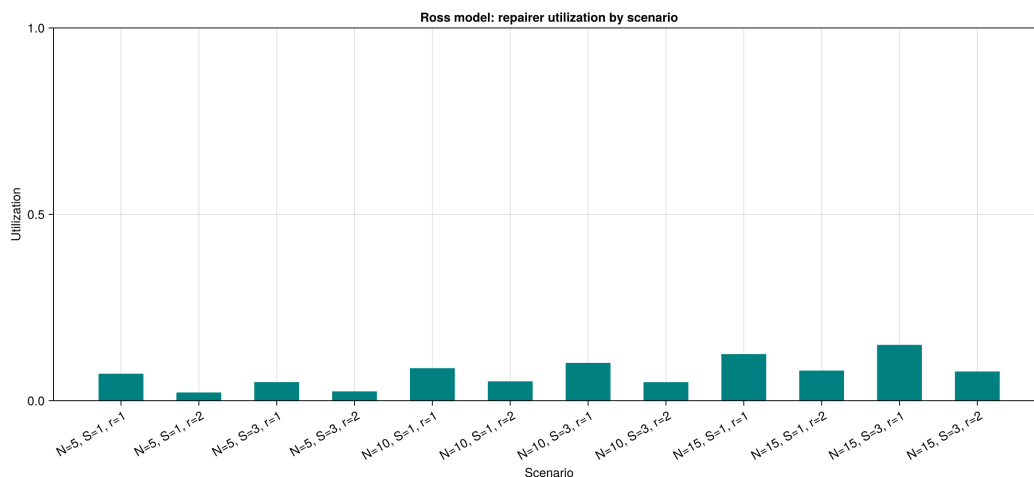


Рисунок 4.22: Загрузка ремонтников по сценариям

Во всех рассмотренных конфигурациях средняя загрузка ремонтников остаётся умеренной. Это подтверждает, что узким местом модели чаще является объём резерва, а не дефицит ремонтных мощностей.

4.9 Генерация `literate`-материалов

Сценарий `scripts/tangle.jl` автоматически формирует производные представления:

```
function main()
    scripts = isempty(ARGS) ? [
        scriptsdir("mmc_literate.jl"),
        scriptsdir("mmc_parameters_literate.jl"),
        scriptsdir("ross_literate.jl"),
        scriptsdir("ross_parameters_literate.jl"),
    ] : ARGS
end
```

```

    for script_path in scripts
        stem = splitext(basename(script_path))[1]
        name = replace(stem, "_literate" => "")
        Literate.script(script_path, scriptsdir(); name = "$(name)_clean", c
        Literate.notebook(script_path, projectdir("notebooks"); name = name
        Literate.markdown(script_path, projectdir("docs"); name = name, cre
    end
end

```

После запуска были получены:

- docs/mmc.md;
- docs/mmc_parameters.md;
- docs/ross.md;
- docs/ross_parameters.md;
- notebooks/mmc.ipynb;
- notebooks/mmc_parameters.ipynb;
- notebooks/ross.ipynb;
- notebooks/ross_parameters.ipynb;
- scripts/mmc_clean.jl;
- scripts/mmc_parameters_clean.jl;
- scripts/ross_clean.jl;
- scripts/ross_parameters_clean.jl.

Вывод сценария `tangle.jl` имеет вид:

```

generated for mmc
generated for mmc_parameters
generated for ross
generated for ross_parameters

```

На Рисунок 4.23 показан фактический запуск `scripts/tangle.jl` с генерацией clean-скриптов, markdown-документов и исполняемых notebook-файлов.

```
lilidji@eiglushchenko:~/labs/lab07/project$ ~/.juliaup/bin/julia --project=. scripts/tangle.jl
[ Info: generating plain script file from `~/labs/lab07/project/scripts/mmc_literate.jl`
[ Info: writing result to `~/labs/lab07/project/scripts/mmc_clean.jl`
[ Info: generating notebook from `~/labs/lab07/project/scripts/mmc_literate.jl`
[ Info: executing notebook `mmc.ipynb`
[ Info: writing result to `~/labs/lab07/project/notebooks/mmc.ipynb`
[ Info: generating markdown page from `~/labs/lab07/project/scripts/mmc_literate.jl`
[ Info: writing result to `~/labs/lab07/project/docs/mmc.md`
generated for mmc
[ Info: generating plain script file from `~/labs/lab07/project/scripts/mmc_parameters_literate.jl`
[ Info: writing result to `~/labs/lab07/project/scripts/mmc_parameters_clean.jl`
[ Info: generating notebook from `~/labs/lab07/project/scripts/mmc_parameters_literate.jl`
[ Info: executing notebook `mmc_parameters.ipynb`
[ Info: writing result to `~/labs/lab07/project/notebooks/mmc_parameters.ipynb`
[ Info: generating markdown page from `~/labs/lab07/project/scripts/mmc_parameters_literate.jl`
[ Info: writing result to `~/labs/lab07/project/docs/mmc_parameters.md`
generated for mmc_parameters
[ Info: generating plain script file from `~/labs/lab07/project/scripts/ross_literate.jl`
[ Info: writing result to `~/labs/lab07/project/scripts/ross_clean.jl`
[ Info: generating notebook from `~/labs/lab07/project/scripts/ross_literate.jl`
[ Info: executing notebook `ross.ipynb`
[ Info: writing result to `~/labs/lab07/project/notebooks/ross.ipynb`
[ Info: generating markdown page from `~/labs/lab07/project/scripts/ross_literate.jl`
[ Info: writing result to `~/labs/lab07/project/docs/ross.md`
generated for ross
[ Info: generating plain script file from `~/labs/lab07/project/scripts/ross_parameters_literate.jl`
[ Info: writing result to `~/labs/lab07/project/scripts/ross_parameters_clean.jl`
[ Info: generating notebook from `~/labs/lab07/project/scripts/ross_parameters_literate.jl`
[ Info: executing notebook `ross_parameters.ipynb`
[ Info: writing result to `~/labs/lab07/project/notebooks/ross_parameters.ipynb`
[ Info: generating markdown page from `~/labs/lab07/project/scripts/ross_parameters_literate.jl`
[ Info: writing result to `~/labs/lab07/project/docs/ross_parameters.md`
generated for ross parameters
```

Рисунок 4.23: Запуск сценария `scripts/tangle.jl`

После генерации была выполнена быстрая проверка состава каталогов `data/`, `plots/`, `docs/` и `notebooks/`. Она подтверждает наличие таблиц, графиков и `literate`-артефактов, необходимых для включения в релиз и в отчётные материалы (Рисунок 4.24).

```
lilidji@eiglushchenko:~/labs/lab07/project$ ls -1 data
ls -1 plots
ls -1 docs
ls -1 notebooks
mmc_customers.csv
mmc_events.csv
mmc_parameter_scan.csv
mmc_summary.csv
ross_events_sample.csv
ross_parameter_scan.csv
ross_runs.csv
ross_summary.csv
mmc_analytic_vs_simulation.png
mmc_busy_servers.png
mmc_queue_length.png
mmc_utilization_heatmap.png
mmc_wait_by_channels.png
mmc_wait_by_lambda.png
mmc_wait_histogram.png
ross_crash_time_by_n.png
ross_crash_time_by_spares.png
ross_crash_time_histogram.png
ross_good_machines.png
```

Рисунок 4.24: Проверка содержимого каталогов результатов

Таким образом, лабораторная работа подготовлена в полностью воспроизводимом формате: исходные сценарии, их чистые версии, markdown-документы и notebook-файлы остаются синхронизированными.

5 Выводы

В лабораторной работе были реализованы две модели дискретно-событийного моделирования. Для системы М/М/с получены журналы заявок и событий, подтверждено согласование базового имитационного сценария с аналитическими характеристиками и показано влияние числа каналов и интенсивности потока на устойчивость системы. Для модели Росса реализованы несколько ремонтников, вычислено время до отказа как имитационно, так и аналитически, а также показано, что ключевую роль в устойчивости системы играет объём резервных машин.

Использование `DrWatson.jl` и `Literate.jl` позволило оформить результат как воспроизводимый вычислительный проект. В итоге подготовлены исходные скрипты, `clean`-версии, `markdown`-документы, `ipynb`-ноутбуки, отчёт и презентация.

6 Приложение. Команды воспроизведения

```
cd ~/labs/lab07/project
~/juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
~/juliaup/bin/julia --project=. -e 'include("scripts/mmc.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/mmc_parameters.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/ross.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/ross_parameters.jl")'
~/juliaup/bin/julia --project=. scripts/tangle.jl
cd ~/labs/lab07/report
quarto render simulation-modeling-lab07-report.qmd --to pdf
quarto render simulation-modeling-lab07-report.qmd --to docx
cd ~/labs/lab07/presentation
quarto render simulation-modeling-lab07-presentation.qmd --to beamer
quarto render simulation-modeling-lab07-presentation.qmd --to revealjs
quarto render simulation-modeling-lab07-presentation.qmd --to pptx
```

1. Gross D., Harris C. M. Fundamentals of Queueing Theory. 3-е изд. New York: Wiley, 1998.
2. Ross S. M. Introduction to Probability Models. 11-е изд. Amsterdam: Academic Press, 2014.

3. Datseris G. и др. DrWatson.jl Documentation. 2024.
4. Ekre F., contributors. Literate.jl Documentation. 2024.
5. Bezanson J. и др. Julia: A Fresh Approach to Numerical Computing. 2017.